IdPGroup

Identification functions for different p-groups

1.0

11 August 2026

Henrik Schanze

Henrik Schanze Email: henrik.schanze@tu-braunschweig.de
Homepage: https://www.iaa.tu-bs.de/henschan
Address: Universitaetsplatz 2

Contents

1 Introduction **3**

2 Functionality **4**
2.1 Identifying groups via a search tree 4
2.2 Loading (strong) siblings 6
2.3 Functions for calculating (strong) siblings 8
2.4 Table of bins of groups 9
2.5 Functions for building a search tree 11

Index **19**

Chapter 1

Introduction

The purpose of the IdPGroup package is to implement identification functions for different p-groups, which are implemented in the SmallGrps library, but lack an effective identification routine in it. The first function identifies the groups of order 512. This is done by calculating a number of invariants of a given group and comparing them with the values of all possible candidates via a precalculated search tree. More details can be found in section 2.1.

Section 2.2 introduces the definitions of (strong) siblings and includes functions to load all such sets of groups of certain order.

Functions that were used to calculate these lists of groups and which can be used to calculate them for different orders can be found in section 2.3.

Once such sets of groups are determined one might want to examine them further by checking certain properties and calculating additionaly invariants. In Section 2.4 one can find functions to calculate some selected invarants and properties for such sets and export them into LaTeX-readable lines of code.

Finally section 2.5 describes all funcions that were used to build the search tree in the descirbed identification routine and which can be used to build other such trees.

Chapter 2

Functionality

This chapter lists all functions contained in the IdPGroup package.

2.1 Identifying groups via a search tree

This section briefly describes the functions and variables of this package associated to the identification of an arbitrary group of order 512 via the search tree `ID_GROUP_512_TREE` (2.1.2). When saying 'identify a group G' we want to find an *id* such that `SmallGroup(512, id)` is isomorphic to G.

The used tree is implement as a nested record with the following components:

- "fp": a list of integers, each being the encoded fingerprint of the group resulting from the specific test performed at the current level of the identification routine. Each integer is gained from appyling `IdPGroup_Pack` (2.5.1) to the fingerprint.
- "next": a list comprised of the respective subbranches corresponding to the fingerprints in "fp". This is either a list of ids or a branch implemented as a record itself.

Depending on the level each record also has one of the following two components:

- "test": a string that indicates which test was used via `TestOnGroup` (2.5.5) to create the listed fingerprints.
- "desc": a list of integers that encode a description of the performed test using `EvalFpCoc`.

The first levels of `ID_GROUP_512_TREE` (2.1.2) try to distinguish groups by the following invariants:

- The rank of the group.
- The collected orders of all elements of the group.
- The numbers of conjugacy classes of each length and representative order (the order of the elements in the class) of the group.
- The isomorphism type of the parent of the group, i.e. the quotient of the group and the smallest non-trivial element of the lower-p-central series.
- The abelian invarants, i.e. the isomorphism types of the quotients of the derived series.

- The cycle structure of the permutations induced on all clusters of conjugacy classes of same length and representative order by the power map $x \mapsto x^3$.

If necessary these invariants are followed by invaraints resulting from tests evaluated on partitions of G.

The initial partition is made of the conjugacy classes of G grouped such that conjugacy classes of the same length and same representative order belong to the same partitioning subset. The results of `EvalFpCoc` applied to a partition give the invaraints used to distinguish groups in these levels of the search tree. The exact behaviour is determined by the content of the component "desc". After the evaluation of each test, the partition is refined according to the result of `EvalFpCoc`.

If the preceding invaraints do not lead to a single group, the following (more time consuming) invaraints are calculated.

- The isomorphism types of the non-trivial factors formed by the group and elements of the smallest non-trivial element of the lower-p-central series.
- The isomorphism types of the non-trivial factors formed by the group and elements of the center.
- The isomorphism types of all maximal subgroups.

Remaining are 139 pairs and 1 triplet of groups of unidentified groups. All these groups are distinguished by a random isomorphism test. That is done by selecting random elements from predefined clusters of conjugacy classes, that were used and potentially refined before, until these elements generate the group. This list of generators is then turned into a `pcgs` and the presentation encoded into an integer by the funtion `CodeGenerators`. The resulting integer modulo 99661 is then compared with precalculated fingerprints of the candidate groups. If none of the fingerprints coincides the process is repeated. Thus the runtime of the identification for these groups is not constant.

2.1.1 IdGroup512

▷ `IdGroup512(`*`G[, opt]`*`)` (function)

Returns: The id of the group of order 512 that is isomorphic to *`G`*.

Identifies the group *`G`* of order 512. This function accepts a record *`opt`* of options with the following content:

- *`tree`*: A search tree that was built using the invariants resulting from *tests*.
- *`anupq`*: A boolean that determines if *IsPqIsomorphicPGroup* from the Anupq-Package should be used to find the to *`G`* isomorphic group among the remaining candidates.
- *`path`*: A boolean that determines if the path through the tree shall be recorded while identification and returned as well.

The default values of *`opt`* is `rec(anupq:=false, tree:=ID_GROUP_512_TREE, path:= false )`.

Example

```
gap> G := Image(IsomorphismPermGroup(SmallGroup(512, 1984)));;
gap> IdGroup512(G);
1984
gap> IdGroup512(G, rec(path := true));
[ 1984, rec( fp := [ 2, 8217, 5321 ], pos := [ 2, 188, 1 ],
```

```
          tests := [ "Rank", "ElmOrds", [ 40511009 ] ] ) ]
gap> H := Image(IsomorphismPermGroup(SmallGroup(512, 449)));;
gap> IdGroup512(H, rec(anupq := false));
449
gap> IdGroup512(H, rec(anupq := true));
449
```

2.1.2 ID_GROUP_512_TREE

▷ ID_GROUP_512_TREE (global variable)

The search tree of groups of order 512 that can determine the vast majority of groups. At the beginning only a small part is loaded. Other branches are loaded if necessary.

2.1.3 LoadTotalTree

▷ LoadTotalTree(*arg*) (function)

Loads the entire search tree ID_GROUP_512_TREE (2.1.2).

2.1.4 IdGroup512Exceptions

▷ IdGroup512Exceptions(*arg*) (function)

Returns: A list of bins of group-ids, that ID_GROUP_512_TREE (2.1.2) can not distinguish without random isomorphism tests.

Example

```
gap> L := IdGroup512Exceptions();;
gap> Collected(List(L, Size));
[ [ 2, 139 ], [ 3, 1 ] ]
gap> L[138];
[ 6249609, 6249610, 6249611 ]
```

2.2 Loading (strong) siblings

This section describes the functions used to load precalculated lists of (strong) siblings, i.e. groups that fulfil the following definitions:

- *subgroup-equivalent groups* are groups with a bijection between the sets of proper subgroups s.t. preimage and image are isomorphic,
- *factor-equivalent groups* are groups with a bijection between the sets of proper factors s.t. preimage and image are isomorphic,
- *siblings* are subgroup- and factor-equivalent groups, where the bijection of proper subgroups preservs conjugacy and maps the Frattini subgroup, lower and upper central series onto the respective counterparts,
- *strong siblings* are siblings that are also Brauer Pairs.

All (strong) siblings of orders dividing 512 together with some trivial cases are implemented in this package and are callable by the following functions.

2.2.1 SiblingsIds

▷ `SiblingsIds(`*`order[, indices]`*`)` (function)

Returns: A list of lists of ids of siblings of size *`order`*.

Loads and returns a list of ids of siblings of size *`order`* if available. Currently available sizes are:

- all orders of the form pq for p and q primes,
- all powers of two smaller than 2^9
- all powers of three smaller than 3^5
- all powers of a prime smaller than p^4
- all products of two powers and
- all orders smaller than 255 except 60, 120, 160, 168, 180, 189, 192 and 240.

If no list is available for the provided order `fail` is returned. If a list *`indices`* is given, only the respective entries are returned.

Example

```
gap> SiblingsIds(101);
[ ]
gap> SiblingsIds(128);
[ [ 1317, 1322 ], [ 1327, 1329 ], [ 1597, 1598 ] ]
gap> SiblingsIds(256, [1..3]);
[ [ 227, 228 ], [ 258, 260, 261 ], [ 845, 846 ] ]
```

2.2.2 Siblings

▷ `Siblings(`*`order[, indices]`*`)` (function)

Returns: A list of siblings of size *`order`*.

Loads and returns a list of siblings of size *`order`* if available. If no list is available for the provided order `fail` is returned. If *`indices`* are given, only the respective entries are returned.

2.2.3 StrongSiblingsIds

▷ `StrongSiblingsIds(`*`order[, indices]`*`)` (function)

Returns: A list of lists of ids of strong siblings of *`order`*.

Loads and returns a list of ids of strong siblings of *`order`* if available. Currently available sizes are:

- all orders of the form pq for p and q primes,
- all powers of two smaller than 2^9
- all powers of three smaller than 3^5
- all powers of a prime smaller than p^4

- all products of two powers and
- all orders smaller than 255 except 60, 120, 160, 168, 180, 189, 192 and 240.

If no list is available for the provided order `fail` is returned. If a list *indices* is given, only the respective entries are returned.

Example

```
gap> StrongSiblingsIds(256);
[ [ 1734, 1735 ], [ 1736, 1737 ], [ 1739, 1740 ], [ 1741, 1742 ],
  [ 3678, 3679 ], [ 4154, 4157 ], [ 4155, 4158 ], [ 4156, 4159 ] ]
gap> StrongSiblingsIds(256, [1..3]);
[ [ 1734, 1735 ], [ 1736, 1737 ], [ 1739, 1740 ] ]
```

2.2.4 StrongSiblings

▷ `StrongSiblings(`*order[, indices]*`)` (function)

Returns: A list of strong siblings of *order*.

Loads and returns a list of strong siblings of *order* if available. If no list is available for the provided order `fail` is returned. If a list *indices* is given, only the respective entries are returned.

2.3 Functions for calculating (strong) siblings

This section describes all functions for calculating (strong) siblings of a given order.

2.3.1 SplitBinsViaTest

▷ `SplitBinsViaTest(`*Bins, order, test*`)` (function)

Returns: A list of sublists of the provided lists in *Bins*.

Takes a list of bins with ids of groups of size *order* and splits them via the outcome of *test* implemented in `TestOnGroup`.

Example

```
gap> SplitBinsViaTest([[1..10]], 512, "MaxSubId");
[ [ 1 ], [ 2 ], [ 3, 8 ], [ 4 ], [ 5 ], [ 6, 7 ], [ 9 ], [ 10 ] ]
```

2.3.2 CalculateSiblings

▷ `CalculateSiblings(`*order[, verbose]*`)` (function)

Returns: A list of lists of ids of groups of size *order* that are siblings.

Determines all siblings among the groups of size *order* by applying `SplitBinsViaTest` (2.3.1) with the tests `"MaxSubId"`, `"SpSubId"`, `"SubId"`, `"MaxFactId"` and `"FactId"` iterativly. The tests `"MaxSubId"` and `"MaxFactId"` are not necessary, but are efficient and eliminate some sets before applying the more costly tests. The boolean *verbose* controls if a collection of bin sizes is printed after each test (`true` as default).

Example

```
gap> CalculateSiblings(128);
Test 'MaxSubId' found following bins of size greater one:[ [ 2, 23 ], [ 3, 2 ] ]
Test 'SpSubId' found following bins of size greater one:[ [ 2, 22 ] ]
Test 'SubId' found following bins of size greater one:[ [ 2, 16 ] ]
Test 'MaxFactId' found following bins of size greater one:[ [ 2, 3 ] ]
```

```
Test 'FactId' found following bins of size greater one:[ [ 2, 3 ] ]
[ [ 1317, 1322 ], [ 1327, 1329 ], [ 1597, 1598 ] ]
gap> CalculateSiblings(128, false);
[ [ 1317, 1322 ], [ 1327, 1329 ], [ 1597, 1598 ] ]
```

2.3.3 DetermineBrauerPairs

▷ `DetermineBrauerPairs(`*`Bins, order`*`)` (function)

Returns: A list of sublists of the provided lists in *`Bins`*.

Determines the Brauer pairs among the bins of ids of groups of size *`order`* listed in *`Bins`*.

2.3.4 CalculateStrongSiblings

▷ `CalculateStrongSiblings(`*`order[, verbose]`*`)` (function)

Returns: A list of lists of ids of groups of size *`order`* that are strong siblings.

First calculats all siblings of size *`order`* via `CalculateSiblings` (2.3.2) and then determines all strong siblings among these by `DetermineBrauerPairs` (2.3.3). The boolean *`verbose`* controls if a collection of bin sizes is printed after each test while siblings are calculated (`true` as default).

Example

```
gap> CalculateStrongSiblings(256, false);
[ [ 1734, 1735 ], [ 1736, 1737 ], [ 1739, 1740 ], [ 1741, 1742 ], [ 3678, 3679 ],
  [ 4154, 4157 ], [ 4155, 4158 ], [ 4156, 4159 ] ]
```

2.4 Table of bins of groups

This section includes the functions of the package that create and fill a table for bins of groups, listing certain invaraints and properties and exporting this table as rows of a latex-tables.

2.4.1 TableOfGroupInvariants

▷ `TableOfGroupInvariants(`*`Bins, order`*`)` (function)

Returns: A list of records.

Calculates some invariants and properties for the groups of order *`order`* with ids in lists of *`bins`*. If the invariants are equal for all groups of a bin, a record listing all information is added to the returned list. The calculated invariants are:

- the rank of the groups,
- the p-class of the groups,
- the size of the groups of automorphisms.

Furthermore the following properties are examined for the groups of a bin:

- Are the groups outer-automorphism-equivaleant, i.e. are the groups of outer-automorphism isomorphic?
- Are the groups isoclinic?

- Are the groups a modular isomorphic pair (MIP), i.e. are their group rings over a field of characteristic p isomorphic?

The last two properties are not examined and left as an empty string in this function, but can be filled via DetermineMIPInTable (2.4.2) and DetermineIsoclinicInTable (2.4.3).

Example

```
gap> Bins := StrongSiblingsIds(512, [1, 5, 14]);
[ [ 1364, 1368 ], [ 1680, 1681 ], [ 15826, 15827 ] ]
gap> TableOfGroupInvariants(Bins, 512);
[ rec( OutEqv := "yes", SizeAut := 8192, ids := [ 1364, 1368 ],
       isoclinic := "",  mip := "", pclass := 5, rank := 2 ),
  rec( OutEqv := "yes", SizeAut := 16384, ids := [ 1680, 1681 ],
       isoclinic := "", mip := "", pclass := 5, rank := 2 ),
  rec( OutEqv := "no", SizeAut := 65536, ids := [ 15826, 15827 ],
       isoclinic := "", mip := "", pclass := 3, rank := 3 ) ]
```

2.4.2 DetermineMIPInTable

▷ DetermineMIPInTable(*tab*) (function)

Returns: A list of records.

Determines if the sets of groups represented by the records in *tab* are modular isomorphic pairs using the ModIsom package. This information is added to each record of *tab* and the updated list is returned. This function is included in an extension of the package and only available if the respective package is loaded.

2.4.3 DetermineIsoclinicInTable

▷ DetermineIsoclinicInTable(*tab*) (function)

Returns: A list of records.

Determines if the sets of groups represented by the records in *tab* are isoclinic using the XMod package. This information is added to each record and the of *tab* updated list is returned. This function is included in an extension of the package and only available if the respective package is loaded.

2.4.4 MergeTablesOfInvariants

▷ MergeTablesOfInvariants(*tab1, tab2*) (function)

Returns: A list of records.

Merges the tables *tab1* and *tab2* into one table. If a record entry of only one is the empty string the non-empty string is used. This enables the parallel computation of e.g. DetermineMIPInTable (2.4.2) and DetermineIsoclinicInTable (2.4.3).

2.4.5 PrintTableToFile

▷ PrintTableToFile(*tab, filename*) (function)

Exports the content of the table *tab* into "*filename*_table.txt" as rows of a latex table.

2.5 Functions for building a search tree

This section describes all functions and variables connected to building a search tree that can be used with `IdGroup512` (2.1.1) to identify groups via implemented invariants.

In short this is done be initializing a tree via calling `TestOnClusterList` (2.5.6) with a selected test on a list of (lists of) ids of groups of given order and then iteratnig the following three steps

- Collecting the clusters of the curent tree via `ClusterList` (2.5.4),
- Performing a choosen test on these clusters via `TestOnClusterList` (2.5.6),
- Extending the tree with these test results via `AddClusterTestResults` (2.5.7).

A very effective set of invaraints can be obtained by calling `CocTestOnClusterList` (2.5.13) in the second step. This should only be done when the groups of each provided cluster yields the same value when the test `CocList` is applied. Furthermore these invaraints should be used for as few as possible clusters since `CocTestOnClusterList` (2.5.13) includes a search for suitable parameters than can take quite some time.

2.5.1 IdPGroup_Pack

▷ `IdPGroup_Pack(`*`list`*`)` (function)

Returns: An interger encoding the content of *`list`*.

Encodes the content of the provided *`list`* into an integer.

Example

```
gap> IdPGroup_Pack([1,2,3]);
123
gap> IdPGroup_Pack([606, 109, 1405]);
63095
gap> IdPGroup_Pack([[1,2,3], [606, 1405]]);
19765
```

2.5.2 ClusterConjClasses

▷ `ClusterConjClasses(`*`G`*`)` (function)

Returns: A list of lists containing the conjugacy classes of *`G`*.

Computes the conjugacy classes of the group *`G`* and clusters them such that each clusters contains all classes of a certain length and certain representative order (meaning the order of any representative).

2.5.3 PowerMapCocCycSt

▷ `PowerMapCocCycSt(`*`coc, n`*`)` (function)

Returns: A lists of lists of lists of integers

Examines the permutation induced by the power map $x \mapsto x^n$ on each cluster of *`coc`*. If a class of a cluster is fixed, the permutation within the class is determined and added to the lists of permutations of the cluster. Finally the cycle structure of each permutation associated to a cluster is examined via `CycleStructurePerm` and saved as a list of integers (undound entries are replaced by 0).

2.5.4 ClusterList

▷ `ClusterList(tree)` (function)

Returns: A list of all clusters of groups in `tree` (i.e. the leafs of `tree`).

Iterates through `tree` and collects information about each leaf as a record. This includes the size of the leaf, the path through `tree` leading to the leaf, the tests performed including descriptions of performed `EvalFpCoc`-tests and the ids of the groups contained in the leaf.

Example

```
gap> tree := rec( fp := [ 63049, 75721, 4310 ],  next := [ [ 3, 8 ], [ 4 ],
gap>            rec( fp := [ 11744, 86646 ], next := [ [ 5 ], [ 6, 7 ] ],
gap>                test := "MaxSubId" ) ], test := "ElmOrds" );;
gap> ClusterList(tree);
[ rec( descl := [  ], ids := [ 3, 8 ], path := [ 63049 ], size := 2,
       tests := [ "ElmOrds" ] ),
  rec( descl := [  ], ids := [ 4 ], path := [ 75721 ], size := 1,
       tests := [ "ElmOrds" ] ),
  rec( descl := [  ], ids := [ 5 ], path := [ 4310, 11744 ], size := 1,
       tests := [ "ElmOrds", "MaxSubId" ] ),
  rec( descl := [  ], ids := [ 6, 7 ], path := [ 4310, 86646 ], size := 2,
       tests := [ "ElmOrds", "MaxSubId" ] ) ]
```

2.5.5 TestOnGroup

▷ `TestOnGroup(G, test[, opt])` (function)

Returns: The result of `test` applied to `G`, if desried encoded as an integer.

Performs the selected `test` on the group `G`. The following choices for `test` are implemented:

- `"ElmOrds"`: Collects the orders of all elements of `G`.
- `"Center"`: Calculates the abelian invariants of the center of `G`.
- `"Rank"`: Calculates the rank of `G`.
- `"AbelInv"`: Computes the order of each term of the derived series and the abelian Invariants of the derived factors of `G`.
- `"UCSId"`: Identifies the proper subgroups of the upper central series of `G` via `IdGroup`.
- `"LCSId"`: Identifies the proper subgroups of the lower central series of `G` via `IdGroup`.
- `"PCSId"`: Identifies the proper subgroups of the p-central series of `G` via `IdGroup`.
- `"SpSubId"`: Computes the Frattini subgroup, the upper and lower central series of `G` and identifies all terms if possible via `IdGroup`.
- `"MaxSubId"`: Collects the sizes and isomorphism types of all maximal subgroups of `G` via `IdGroup`.
- `"SubId"`: Collects the sizes and isormorphism types of conjugacy classes of all proper subgoups via `IdGroup`.
- `"MaxFactId"`: Collects the sizes and isomorphism types of all maximal factors of `G` via `IdGroup`.

- *"CentQuot"*: Collects the sizes and isomorphism types of all quotients of *G* with non-trivial elements of the center of *G* via IdGroup.
- *"LCSFactId"*: Collects the isomorphism types of all quotients of *G* with with cyclic subgroups generated by by non-trivial elements of the last non-trivial term of the p-central series.
- *"FactId"*: Identifies all proper factors of *G* via IdGroup.
- *"CocList"*: Collects the number of conjugacy classes of a *G* of same length and same representative order (the order of a representative).
- *"PMCycSt"*: Determines the cycle-structure invariant for the power map $x \mapsto x^3$ on the clusters of conjugacy classes via PowerMapCocCycSt.
- *"OutId"*: Tries to identify the groups of outer automorphisms of *G*, either via IdGroup (if available) or via its size and TestOnGroup with *"ElmOrds"* and *"CocList"* (*"MaxSubId"* and *"CentQuot"* can also be used if feasible).
- *"LattEqv"*: Calculates all conjugacy classes of proper subgoups of *G*, stores their size, identifies the the isomorphism types of each class and determines the isomorphism types and conjugacy classes in *G* of its maximal subgroups and minimal supergroups.

This function accepts a record *opt* of options with the single content *opt.pack*, a boolean that determines if IdPGroup_Pack should be applied to the result of a test. The default value of *opt* is *rec*(*pack* :=true).

Example

```
gap> G := SmallGroup(512, 1405);;
gap> TestOnGroup(G, "ElmOrds");
85416
gap> TestOnGroup(G, "ElmOrds", rec(pack := false));
[ [ 1, 1 ], [ 2, 7 ], [ 4, 24 ], [ 8, 288 ], [ 16, 64 ], [ 32, 128 ] ]
```

2.5.6 TestOnClusterList

▷ TestOnClusterList(*CL, test[, opt]*) (function)

Returns: Either a list of pairs each containg the path to the respective cluster (which is a leaf of a search tree) and the subbranch resulting from the results of *test*, or just the list of subbranches if *CL* contains only lists of integers. If *CL* consits of a single list of integers the single resulting subbranch is returned on its own.

Performs the choosen test on the groups of each record in the *CL*. The tests are performed by TestOnGroup (2.5.5). *CL* could either be a list of records produced by ClusterList (2.5.4) or be a list of lists containing ids. In the former case the output is a list of pairs containg the path of a cluster and the subbranch resulting from the test. In the latter case only a list of subbranches is returned. If in the latter case the returned list only one contains one subbranch, the subbranch itself is returned. This is useful when building the first level of a search tree. This function accepts a record *opt* of options with the following content:

- *pack*: A boolean that is handed to TestOnGroup (2.5.5).
- *ign1cl*: A boolean that determines if clusters of size 1 should be ignored.

- *order*: An integer giving the order of the groups with ids in *CL*.
- *verbose*: A boolean that determines if progress should be printed.
- *notetest*: A boolean that decides if the performed test should be noted in the returned subbranches. If so subbranches, that do not split a cluster are omited.

The default value of *opt* is *rec*(*pack*:=true, *ign1cl*:=true, *order*:=512, *verbose*:=true, *notetest*:=true).

Example

```
gap> TestOnClusterList([[1..10]],"ElmOrds", rec(verbose:=false));
rec( fp := [ 88570, 4310, 63049, 75721 ],
     next := [ [ 1 ], [ 2, 5, 6, 7, 9, 10 ], [ 3, 8 ], [ 4 ] ],
     test := "ElmOrds")
gap> TestOnClusterList([[1,2,5],[7,8,9]],"ElmOrds", rec(verbose:=false));
[ rec( fp := [ 88570, 4310 ], next := [ [ 1 ], [ 2, 5 ] ],
       test := "ElmOrds" ),
  rec( fp := [ 4310, 63049 ], next := [ [ 7, 9 ], [ 8 ] ],
       test := "ElmOrds") ]
gap> tree := TestOnClusterList([[3..8]],"ElmOrds", rec(verbose:=false));;
gap> CL := ClusterList(tree);
[ rec( descl := [  ], ids := [ 3, 8 ], path := [ 63049 ], size := 2,
       tests := [ "ElmOrds" ]),
  rec( descl := [  ], ids := [ 4 ], path := [ 75721 ], size := 1,
       tests := [ "ElmOrds" ]),
  rec( descl := [  ], ids := [ 5, 6, 7 ], path := [ 4310 ], size := 3,
       tests := [ "ElmOrds" ]) ]
gap> TestOnClusterList(CL, "MaxSubId", rec(verbose:=false, notetest := false));
[ [ [ 63049 ], rec( fp := [ 62603 ], next := [ [ 3, 8 ] ] ) ],
  [ [ 4310 ], rec( fp := [ 11744, 86646 ], next := [ [ 5 ], [ 6, 7 ] ] ) ] ]
```

2.5.7 AddClusterTestResults

▷ AddClusterTestResults(*tree*, *TR*) (function)

Returns: The extended *tree* with the subbranches in *TR*.

Extends *tree* by the test results stored in *TR*, a list of pairs each containing the path in *tree* and a branch (a tree of depth one), extending the corresponding cluster of the inital *tree*.

Example

```
gap> tree := TestOnClusterList([[3..8]],"ElmOrds", rec(verbose:=false));
rec( fp := [ 63049, 75721, 4310 ], next := [ [ 3, 8 ], [ 4 ], [ 5, 6, 7 ] ],
     test := "ElmOrds")
gap> TR:=TestOnClusterList(ClusterList(tree),"MaxSubId",rec(verbose:=false));
[ [ [ 4310 ], rec( fp := [ 11744, 86646 ], next := [ [ 5 ], [ 6, 7 ] ],
                   test := "MaxSubId" ) ] ]
gap> AddClusterTestResults(tree,TR);
rec( fp := [ 63049, 75721, 4310 ],
     next := [ rec( fp := [ 62603 ], next := [ [ 3, 8 ] ] ), [ 4 ],
               rec( fp := [ 11744, 86646 ], next := [ [ 5 ], [ 6, 7 ] ],
     test := "MaxSubId") ], test := "ElmOrds" )
```

2.5.8 BuildSearchTree

▷ `BuildSearchTree(N, order, tests)` (function)

Returns: A search tree (implemented as a record).

Builds a search tree for the groups with ids in `N` or, if `N` is an integer, the first `N` groups of size `order`, using `TestOnClusterList` (2.5.6) with the tests from the list `tests`.

Example

```
gap> BuildSearchTree(5,512,["ElmOrds", "MaxSubId"]);
rec( fp := [ 88570, 4310, 63049, 75721 ], next := [ [ 1 ],
     rec( fp := [ 76646, 11744 ], next := [ [ 2 ], [ 5 ] ],
     test := "MaxSubId" ), [ 3 ], [ 4 ] ], test := "ElmOrds" )
gap> BuildSearchTree([1,2,3,5,7], 512, ["ElmOrds", "CocList"]);
rec( fp := [ 88570, 4310, 63049 ], next := [ [ 1 ],
     rec( fp := [ 96561, 32173 ], next := [ [ 2 ], [ 5, 7 ] ],
     test := "CocList" ), [ 3 ] ], test := "ElmOrds" )
```

2.5.9 FindParametersForCocTest

▷ `FindParametersForCocTest(Coc, paralist, it, maxit[, opt])` (function)

Returns: A pair made of a list of return values from `EvalFpCoc` and a list describing the test performed by `EvalFpCoc`.

Does a parameter search such that evaluating `EvalFpCoc` on the clusters of conjugacy classes in `coc` results in at least two different values. If no suitable parameters are found, the function calls itself iterativly on parameters that alter `Coc` until either a suitable list of parameters is found or `it` is equal to `maxit`. The parameters of the current iteration are provided in the list `paralist`. If the function is initialized with `it`=0, it only returns a list of parameters that alter the clusters in `Coc`. Depending on the size of `coc` their can be a lot of parameters to check. To decrease the needed time the following configurations are skipped:

- $[1,i,j]$, where the representative order of `coc[i]` and j are not coprime.
- $[2,i,j]$, where the representative order of `coc[i]` is less or equal then the order of `coc[j]` or the length of the classes in `coc[i]` is less then the ones of `coc[j]`.
- $[3,i,j]$, where the representative order of `coc[i]` is larger or equal then the order of `coc[j]` or the length of the classes in `coc[i]` is larger then the ones of `coc[j]`.
- $[4,i,j,k]$, where either the classes in `coc[i]` or `coc[j]` are of length one.
- $[5,i,j,k]$, where either the classes in `coc[i]` or `coc[j]` are of length one.

This function accepts a record `opt` of options with the following content:

- `target`: A list of integers that determines the desired second entries of the parameters.
- `maxlevel`: An integer that determines the maximal first parameter entry. That might be useful, since the parameters for test 4 and 5 often need more time then the rest.

The default value of `opt` is $rec(target := [1..Size(Coc[1])], maxlevel := 5$.

Example

```
gap> Coc:=List([3, 8],id->ClusterConjClasses(SmallGroup(512, id)));;
gap> FindParametersForCocTest(Coc, [], 1, 1);
[ [ 1363, 46760 ], [ 3, 2, 4 ] ]
gap> FindParametersForCocTest(Coc, [], 1, 1, rec(target:=[4]));
[ [ 63010, 76026 ], [ [ 4, 4, 6, 5 ] ] ]
gap> FindParametersForCocTest(Coc, [], 0, 1, rec(target:=[4]));
[ [ 3, 4, 6 ], [ 4, 4, 6, 3 ], [ 4, 4, 6, 4 ], [ 4, 4, 6, 5 ] ]
gap> FindParametersForCocTest(Coc, [], 0, 1, rec(target:=[4], maxlevel := 3));
[ [ 3, 4, 6 ] ]
gap> Coc:=List([1883, 1885],id->ClusterConjClasses(SmallGroup(512, id)));;
gap> FindParametersForCocTest(Coc, [], 1, 2);
[ [ 8, 18 ], [ [ 1, 9, 3 ], [ 2, 14, 9 ] ] ]
```

2.5.10 DeEncodeDesc

▷ DeEncodeDesc(*desc*) (function)

Returns: An interger or a list of integers

De- or encodes a description for EvalFPCoc, depending on if an integer (decodes to a list) or a list of integers (encodes to an integer) is provided.

Example

```
gap> DeEncodeDesc([3,2,4]);
302004
gap> DeEncodeDesc(302004);
[3, 2, 4]
```

2.5.11 AlterCocAccordingToFp

▷ AlterCocAccordingToFp(*coc*, *desc*, *fp*) (function)

Returns: A list of lists containing the conjugacy classes of a group.

Alters the clusters of conjugacy classes *coc* according the result *fp* from EvalFPCoc evaluated with *desc* by spliting the cluster at position *desc[2]* of *coc* into smaller ones.

Example

```
gap> coc := ClusterConjClasses(SmallGroup(512, 3));;
gap> List(coc, Size);
[ 1, 15, 4, 24, 12, 48 ]
gap> fp := EvalFpCoc(coc, [3,2,4]);
[ 0, 0, 16, 16, 0, 0, 0, 0, 0, 16, 0, 0, 0, 0, 0 ]
gap> cocnew := AlterCocAccordingToFp(coc,[3,2,4],fp);;
gap> List(cocnew, Size);
[ 1, 12, 3, 4, 24, 12, 48 ]
```

2.5.12 CocSplitRoutine

▷ CocSplitRoutine(*Ids*, *order*, *desclist*, *iterate*) (function)

Returns: A search tree (implemented as a record).

Tries to split the list *Ids* of group ids of *order* with ids as far as possible on the returned values of EvalFpCoc and returns the resulting subbranch for a search tree with the relevant details. The parameter search is done with at most *iterate* iterations. Past perfomances of EvalFpCoc which altered

the clusters of conjugacy classes of the groups can be recreated in advance via the list *desclist*. If no such splitis possible a warning is printed and *Ids* is returned.

Example

```
gap> CocSplitRoutine([5,6,7], 512, [], 1);
rec( desc := [104003], fp := [ 1214, 16 ], next := [ [ 5 ],
     rec( desc := [40204005], fp := [ 20792, 46881 ],
          next := [ [ 6 ], [ 7 ] ] ) ] )
gap> CocSplitRoutine([1529,1530,1531], 512, [], 2);
rec( desc := [ 209003 ], fp := [ 4, 14 ], next :=
   [ rec( desc := [ 107003, 212007 ], fp := [ 14, 618 ],
     next := [ [ 1529 ], [ 1530 ] ] ), [ 1531 ] ] )
```

2.5.13 CocTestOnClusterList

▷ `CocTestOnClusterList(`*CL*`[, `*opt*`])` (function)

Returns: A list of pairs each containing the path of the respective cluster and the subbranch resulting from `CocSplitRoutine`.

Tries to split each cluster in *CL* via `CocSplitRoutine` (2.5.12). *CL* must be a cluster list produced by `ClusterList` (2.5.4). This function accepts a record *opt* of options with the following content:

- *pack*: A boolean that determines if `IdPGroup_Pack` (2.5.1) should be applied to the result.
- *order*: An integer giving the order of the groups with ids in *CL*.
- *iterate*: An integer that determines the maximal number of iterations done while searching parameters.
- *verbose*: An boolean that determines if progress should be printed.

The default value of *opt* is *rec(pack*:=true, *order*:=512, *iterate*:=1,*verbose*:=true*)*.

Example

```
gap> tree := TestOnClusterList([[3,5,6,7,8]],"CocList",rec(verbose:=false));;
gap> CL := ClusterList(tree);
[ rec( descl := [  ], ids := [ 3, 8 ], path := [ 337 ], size := 2,
       tests := [ "CocList" ] ),
  rec( descl := [  ], ids := [ 5, 6, 7 ], path := [ 32173 ], size := 3,
       tests := [ "CocList" ] ) ]
gap> TR := CocTestOnClusterList(CL,rec(verbose:=false));;
gap> AddClusterTestResults(tree, TR);;
gap> ClusterList(tree);
[ rec( descl := [ [ 302004 ] ], ids := [ 3 ], path := [ 337, 1363 ], size := 1,
       tests := [ "CocList", "" ]),
  rec( descl := [ [ 302004 ] ], ids := [ 8 ], path := [ 337, 46760 ], size := 1,
       tests := [ "CocList", ""  ]),
  rec( descl := [ [ 104003 ] ], ids := [ 5 ], path := [ 32173, 1214 ],
       size := 1, tests := [ "CocList", ""  ] ),
  rec( descl := [ [ 104003 ], [ 40204005 ] ], ids := [ 6 ],
       path := [ 32173, 16, 20792 ], size := 1, tests := [ "CocList", ""  ] ),
  rec( descl := [ [ 104003 ], [ 40204005 ] ], ids := [ 7 ],
       path := [ 32173, 16, 46881 ], size := 1, tests := [ "CocList", ""  ] ) ]
```

2.5.14 ParametersRandPresentTest

▷ `ParametersRandPresentTest(`*`CL, order, maxl`*`)` (function)

Returns: A list consisting of pairs of a list of ids and a record containing the parameters for a random presentation test via `ID_GROUP_FUNCS[18]`.

Determines the needed parameters for the groups of size *`order`* in the clusters *`cl`* of *`CL`*. Creates the clusters of conjugcy classes and alters them accordingly to the *`cl.descl`*. Afterwards minimal generators for all groups and their position among the clusters of conjugacy classes. If the positions do not coincide, the positions of the generators of the first group are selected and the generators of the other groups are selected from the same clusters. Otherwise the positions and generators are sorted by the positions. Sequently the function tries to refine the corresponding clusters to limit the choices to choose from and repeat the process until either the number of refinments is equal or larger than *`maxl`* or no more refinments could be found.

Example

```
gap> ParametersRandPresentTest([rec(descl:=[], ids:=[1367, 1371])], 512, 0);
[ [ [ 1367, 1371 ], rec( descl := [  ],
     fp := [ 39080, 80814 ], func := 18, pos := [ 8, 10 ] ) ] ]
gap> ParametersRandPresentTest([rec(descl:=[], ids:=[1367, 1371])], 512, 1);
[ [ [ 1367, 1371 ], rec( descl := [ 106003 ],
     fp := [ 39080, 80814 ], func := 18, pos := [ 9, 11 ] ) ] ]
```

Index

AddClusterTestResults, 14
AlterCocAccordingToFp, 16

BuildSearchTree, 15

CalculateSiblings, 8
CalculateStrongSiblings, 9
ClusterConjClasses, 11
ClusterList, 12
CocSplitRoutine, 16
CocTestOnClusterList, 17

DeEncodeDesc, 16
DetermineBrauerPairs, 9
DetermineIsoclinicInTable, 10
DetermineMIPInTable, 10

FindParametersForCocTest, 15

IdGroup512, 5
IdGroup512Exceptions, 6
IdPGroup_Pack, 11
ID_GROUP_512_TREE, 6

LoadTotalTree, 6

MergeTablesOfInvariants, 10

ParametersRandPresentTest, 18
PowerMapCocCycSt, 11
PrintTableToFile, 10

Siblings, 7
SiblingsIds, 7
SplitBinsViaTest, 8
StrongSiblings, 8
StrongSiblingsIds, 7

TableOfGroupInvariants, 9
TestOnClusterList, 13
TestOnGroup, 12